

CS3301 - Data Structures

Topic: array

1. SUMMARY

An array is a data structure that stores a collection of elements of the same data type in contiguous memory locations. It is a linear data structure, meaning that elements are stored in a sequence or an order. The array is a fundamental data structure in computer science and is used extensively in programming. The reference describes arrays as a collection of elements arranged in a sequential manner, either in contiguous memory locations or using pointers or links to associate elements sequentially. The array can be classified into different types, including one-dimensional, multi-dimensional, and jagged arrays. The reference also discusses the applications of arrays, such as in sorting and searching algorithms, and highlights the advantages and disadvantages of using arrays. The key principles of arrays include the concept of indexing, where each element is assigned a unique index, and the concept of bounds checking, where the programmer checks if the index is within the valid range of the array. The reference mentions that arrays are dynamic data structures, meaning that they can be resized at runtime. The array is used extensively in real-world applications, such as in databases, file systems, and web development.

2. SHORT NOTES

Definition and Concept

An array is a data structure that stores a collection of elements of the same data type in contiguous memory locations. It is a linear data structure, meaning that elements are stored in a sequence or an order. The array can be thought of as a container that holds a fixed-size collection of elements.

Types and Classifications

There are several types of arrays, including:

- One-dimensional arrays: These are the simplest type of array and store elements in a single row.
- Multi-dimensional arrays: These store elements in multiple rows and columns.
- Jagged arrays: These are arrays of arrays, where each inner array can have a different length.

Step-by-Step Working Principle

Here is a step-by-step explanation of how arrays work:

1. The array is declared with a specific data type and size.
2. The programmer assigns values to the elements of the array.
3. The programmer can access and manipulate the elements of the array using their index.
4. The array is stored in contiguous memory locations, which allows for fast access and manipulation.

Important Formulas and Equations

There are no specific formulas or equations for arrays, but the following is a relevant concept:

- Indexing: Each element of the array is assigned a unique index, which is used to access and manipulate the element.

Diagrams Description

Here is a simple diagram of an array:

...

| Element 1 | Element 2 | Element 3 | ... | Element N |

...

Advantages and Disadvantages

Advantages of arrays:

- Fast access and manipulation of elements
- Efficient use of memory
- Dynamic resizing at runtime

Disadvantages of arrays:

- Fixed-size array can lead to memory waste
- Out-of-bounds error if index is not within valid range

Real-World Applications

Arrays are used extensively in real-world applications, such as:

- Databases: Arrays are used to store and manage large datasets.
- File systems: Arrays are used to store and manage files and directories.
- Web development: Arrays are used to store and manage user input and data.

Comparison with Related Concepts

Arrays can be compared with other data structures, such as:

- Linked lists: Arrays store elements in contiguous memory locations, while linked lists store elements in a sequence of nodes.
- Stacks and queues: Arrays can be used to implement stacks and queues, but they are not the most efficient data structures for these operations.

Time and Space Complexity

The time and space complexity of arrays depends on the specific operation being performed. Here are some

examples:

- Accessing an element: $O(1)$
- Inserting or deleting an element: $O(n)$
- Searching for an element: $O(n)$

The space complexity of an array is $O(n)$, where n is the size of the array.

Terminology

Some important terminology related to arrays includes:

- Indexing: The process of assigning a unique index to each element of the array.
- Bounds checking: The process of checking if the index is within the valid range of the array.
- Dynamic resizing: The ability to resize the array at runtime.

3. CODING EXAMPLES

C Implementation

```
```c
#include <stdio.h>

int main() {
 int arr[5];
 arr[0] = 10;
 arr[1] = 20;
 arr[2] = 30;
 arr[3] = 40;
 arr[4] = 50;
```

```
printf("Element at index 2: %d\n", arr[2]);
printf("Element at index 5: %d\n", arr[5]); // out-of-bounds error

return 0;
}
...
```

### Python Implementation

```
```python  
class Array:  
    def __init__(self, size):  
        self.size = size  
        self.elements = [None] * size  
  
    def set_element(self, index, value):  
        if index < 0 or index >= self.size:  
            raise IndexError("Index out of bounds")  
        self.elements[index] = value  
  
    def get_element(self, index):  
        if index < 0 or index >= self.size:  
            raise IndexError("Index out of bounds")  
        return self.elements[index]  
  
arr = Array(5)  
arr.set_element(0, 10)  
arr.set_element(1, 20)  
arr.set_element(2, 30)  
arr.set_element(3, 40)  
arr.set_element(4, 50)
```

```
print(arr.get_element(2))  
print(arr.get_element(5)) # out-of-bounds error  
...
```

Algorithm

1. Initialize an empty array.
2. Set the size of the array.
3. Assign values to the elements of the array using their index.
4. Access and manipulate the elements of the array using their index.

Time and Space Complexity

- Best Case: $O(1)$
- Average Case: $O(n)$
- Worst Case: $O(n)$
- Space Complexity: $O(n)$

4. IMPORTANT QUESTIONS

Q1. (2 marks) What is an array and how is it different from a linked list?

Answer: An array is a data structure that stores a collection of elements of the same data type in contiguous memory locations. It is different from a linked list in that elements are stored in contiguous memory locations, whereas in a linked list, elements are stored in a sequence of nodes.

Q2. (13 marks) Explain the concept of indexing and bounds checking in arrays.

Answer: Indexing is the process of assigning a unique index to each element of the array. The index is used to access and manipulate the element. Bounds checking is the process of checking if the index is within the

valid range of the array. If the index is out of bounds, an error is raised.

Q3. (15 marks) Compare and contrast arrays with linked lists.

Answer: Arrays store elements in contiguous memory locations, whereas linked lists store elements in a sequence of nodes. Arrays are more efficient for accessing and manipulating elements, but linked lists are more suitable for inserting and deleting elements.

Q4. (2 marks) What is the time and space complexity of accessing an element in an array?

Answer: The time complexity is $O(1)$ and the space complexity is $O(1)$.

Q5. (13 marks) Explain the concept of dynamic resizing in arrays.

Answer: Dynamic resizing is the ability to resize the array at runtime. This can be done by allocating new memory and copying the elements to the new memory.

Q6. (15 marks) Compare and contrast multi-dimensional arrays with jagged arrays.

Answer: Multi-dimensional arrays store elements in multiple rows and columns, whereas jagged arrays are arrays of arrays, where each inner array can have a different length.

Q7. (2 marks) What is the time and space complexity of searching for an element in an array?

Answer: The time complexity is $O(n)$ and the space complexity is $O(1)$.

Q8. (13 marks) Explain the concept of out-of-bounds error in arrays.

Answer: An out-of-bounds error occurs when the index is not within the valid range of the array. This can lead to unexpected behavior and errors.

Q9. (15 marks) Compare and contrast arrays with stacks and queues.

Answer: Arrays can be used to implement stacks and queues, but they are not the most efficient data structures for these operations.

Q10. (2 marks) What is the time and space complexity of inserting or deleting an element in an array?

Answer: The time complexity is $O(n)$ and the space complexity is $O(1)$.

Q11. (13 marks) Explain the concept of indexing in arrays.

Answer: Indexing is the process of assigning a unique index to each element of the array. The index is used to access and manipulate the element.

Q12. (15 marks) Compare and contrast arrays with linked lists in terms of memory usage.

Answer: Arrays store elements in contiguous memory locations, which can lead to memory waste if the array is not fully utilized. Linked lists store elements in a sequence of nodes, which can lead to more efficient memory usage.

Q13. (2 marks) What is the time and space complexity of creating an array?

Answer: The time complexity is $O(1)$ and the space complexity is $O(n)$.

Q14. (13 marks) Explain the concept of bounds checking in arrays.

Answer: Bounds checking is the process of checking if the index is within the valid range of the array. If the index is out of bounds, an error is raised.

Q15. (15 marks) Compare and contrast arrays with linked lists in terms of insertion and deletion operations.

Answer: Arrays are more efficient for accessing and manipulating elements, but linked lists are more suitable for inserting and deleting elements.

5. MCQs

Q1. What is an array?

- a) A data structure that stores a collection of elements of the same data type in contiguous memory locations.
- b) A data structure that stores a collection of elements of different data types in contiguous memory locations.
- c) A data structure that stores a collection of elements of the same data type in a sequence of nodes.
- d) A data structure that stores a collection of elements of different data types in a sequence of nodes.

Answer: a) A data structure that stores a collection of elements of the same data type in contiguous memory locations.

Explanation: An array is a data structure that stores a collection of elements of the same data type in contiguous memory locations.

Q2. What is the time complexity of accessing an element in an array?

- a) $O(1)$
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n \log n)$

Answer: a) $O(1)$

Explanation: The time complexity of accessing an element in an array is $O(1)$ because the element can be accessed directly using its index.

Q3. What is the space complexity of an array?

- a) $O(1)$
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n \log n)$

Answer: b) $O(n)$

Explanation: The space complexity of an array is $O(n)$ because the array stores a collection of elements of size n .

Q4. What is the time complexity of searching for an element in an array?

- a) $O(1)$
- b) $O(n)$

- c) $O(n^2)$
- d) $O(n \log n)$

Answer: b) $O(n)$

Explanation: The time complexity of searching for an element in an array is $O(n)$ because the search algorithm must compare each element with the target element.

Q5. What is the time complexity of inserting or deleting an element in an array?

- a) $O(1)$
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n \log n)$

Answer: b) $O(n)$

Explanation: The time complexity of inserting or deleting an element in an array is $O(n)$ because the algorithm must shift all elements after the insertion or deletion point.

Q6. What is the time complexity of creating an array?

- a) $O(1)$
- b) $O(n)$